

# Chapter 12

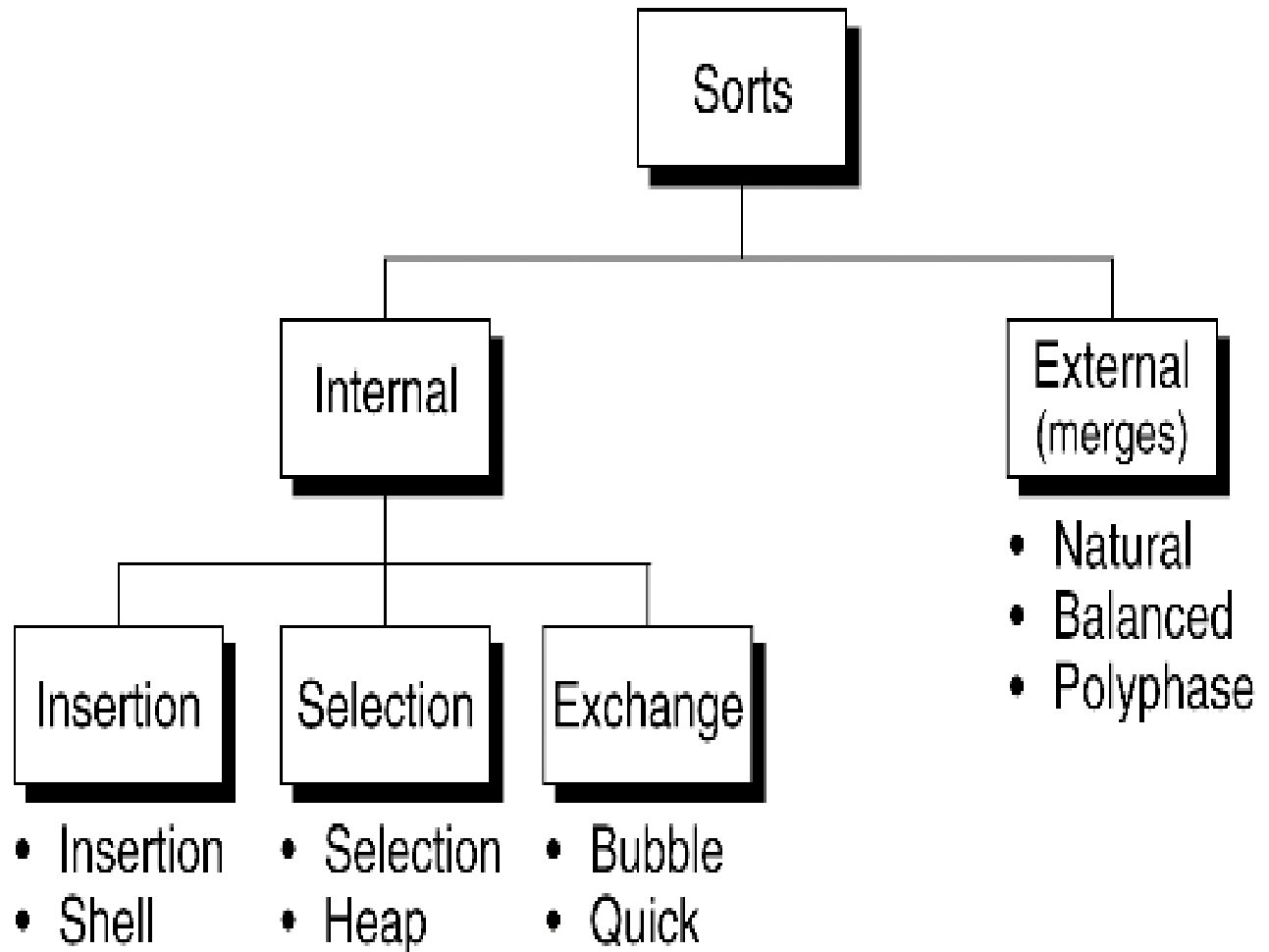
## ***Sorting***

### **Objectives**

---

***Upon completion you will be able to:***

- **Understand the basic concepts of internal sorts**
- **Discuss the relative efficiency of different sorts**
- **Recognize and discuss selection, insertion and exchange sorts**



**FIGURE 12-1** Sort Classifications

365 blue  
212 green  
876 white  
212 yellow  
119 purple  
737 green  
212 blue  
443 red  
567 yellow

**(a) Unsorted data**

119 purple  
212 green  
212 yellow  
212 blue  
365 blue  
443 red  
567 yellow  
737 green  
876 white

**(b) Stable sort**

119 purple  
212 blue  
212 green  
212 yellow  
365 blue  
443 red  
567 yellow  
737 green  
876 white

**(c) Unstable sort**

---

**FIGURE 12-2** Sort Stability

# 12-2 Selection Sorts

*In each pass of the selection sort, the smallest element is selected from the unsorted sublist and exchanged with the element at the beginning of the unsorted list. We discuss two classic selection sorts, straight selection and heap sort.*

- Straight Selection Sort
- Heap Sort
- Selection Sort Efficiency
- Selection Sort Implementation

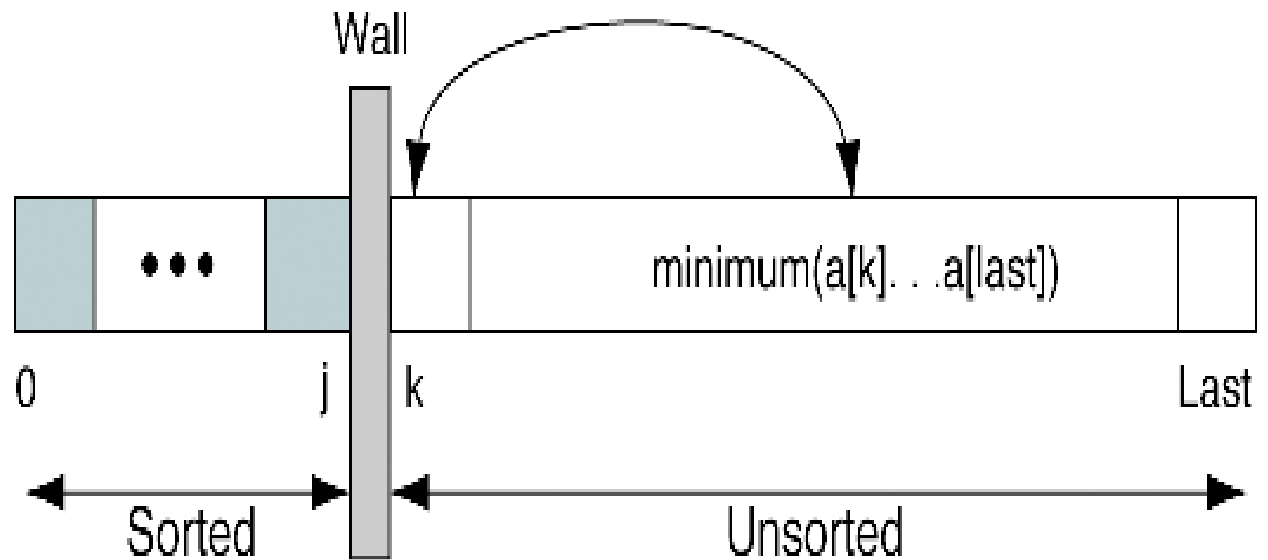
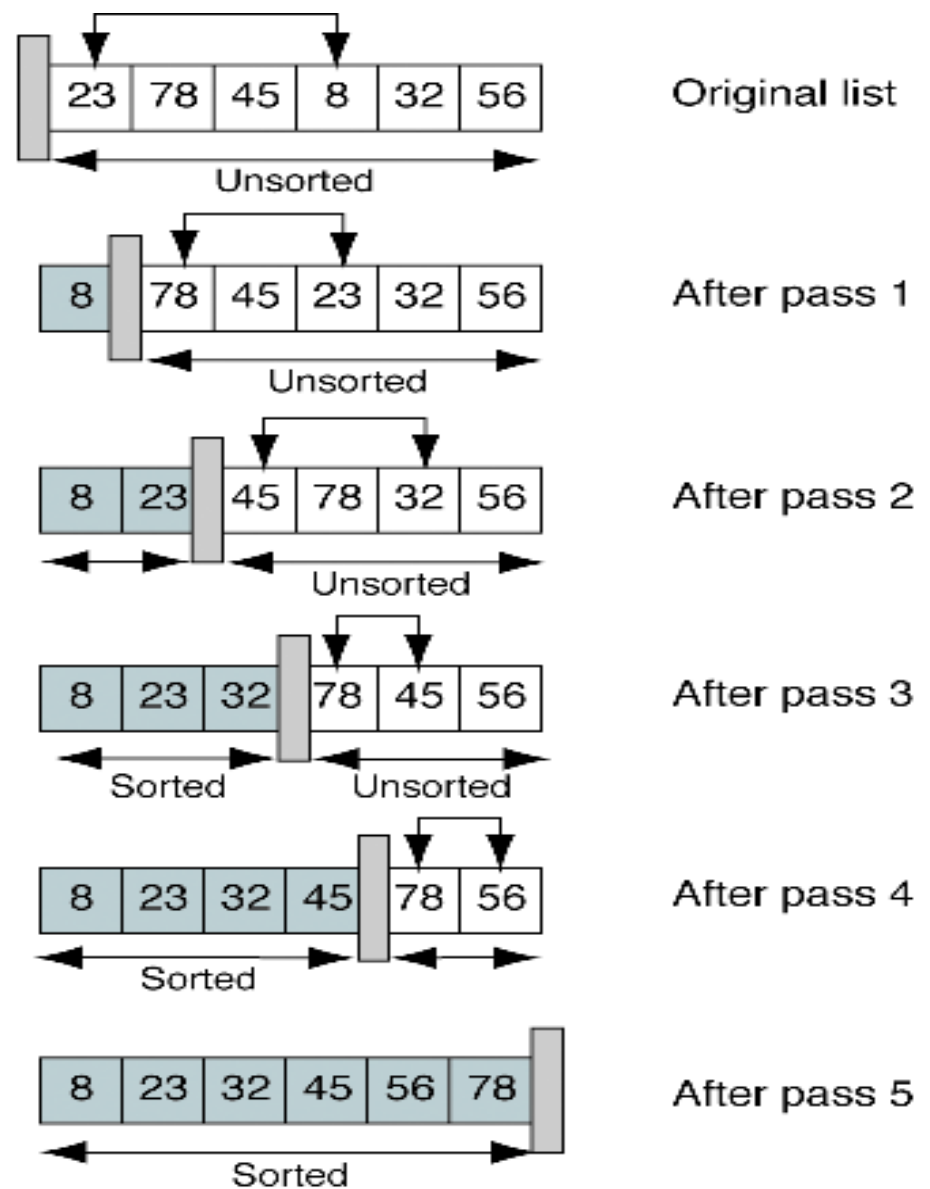


FIGURE 12-3 Selection Sort Concept



**FIGURE 12-4** Selection Sort Example

## ALGORITHM 12-1 Selection Sort

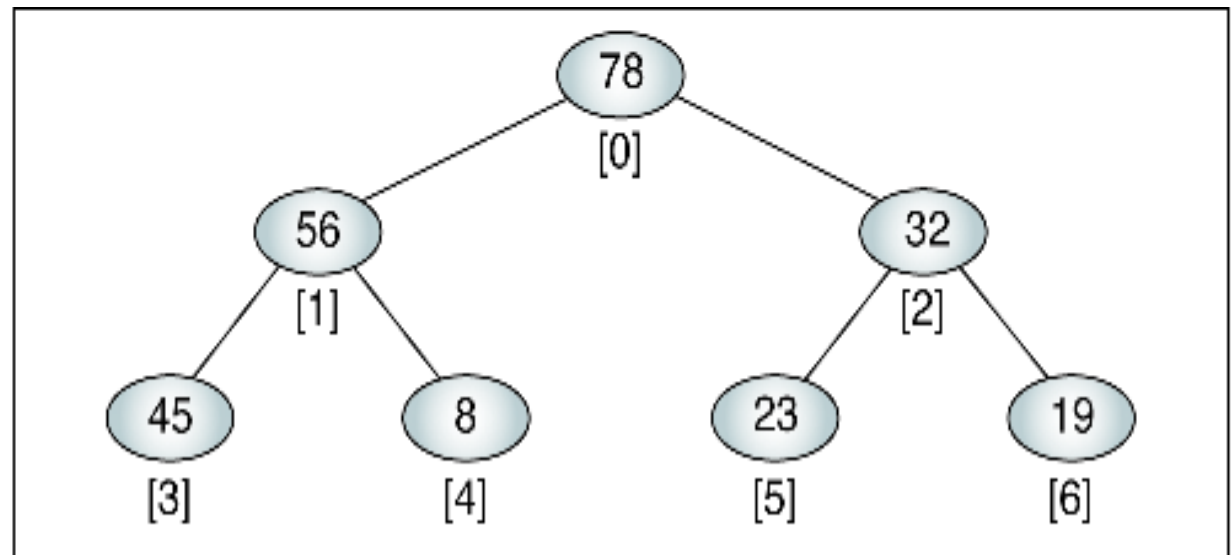
```
Algorithm selectionSort (list, last)
Sorts list array by selecting smallest element in
unsorted portion of array and exchanging it with element
at the beginning of the unsorted list.
    Pre  list must contain at least one item
         last contains index to last element in the list
    Post list has been rearranged smallest to largest
1  set current to 0
2  loop (until last element sorted)
    1  set smallest to current
```

*continued*

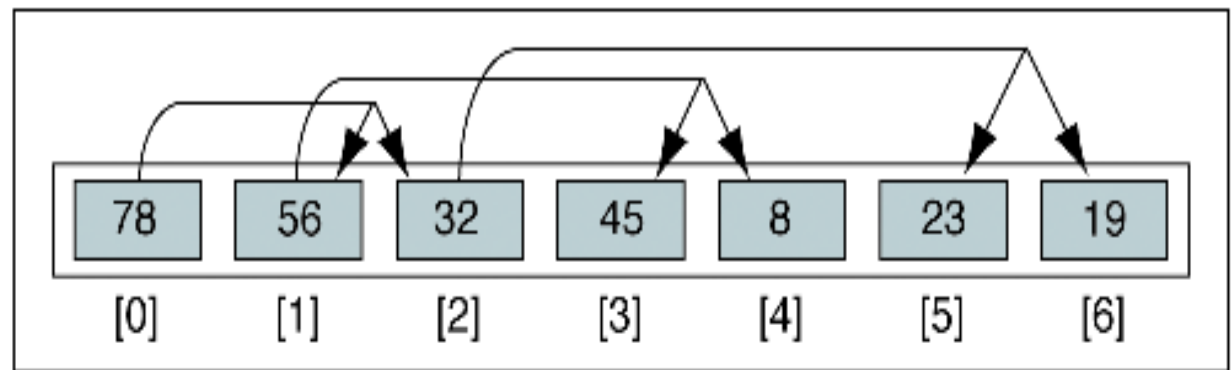
## ALGORITHM 12-1 Selection Sort (continued)

```
2  set walker    to current + 1
3  loop (walker <= last)
    1  if (walker key < smallest key)
        1  set smallest to walker
    2  increment walker
4  end loop
   Smallest selected: exchange with current element.
5  exchange (current, smallest)
6  increment current
3  end loop
end selectionSort
```



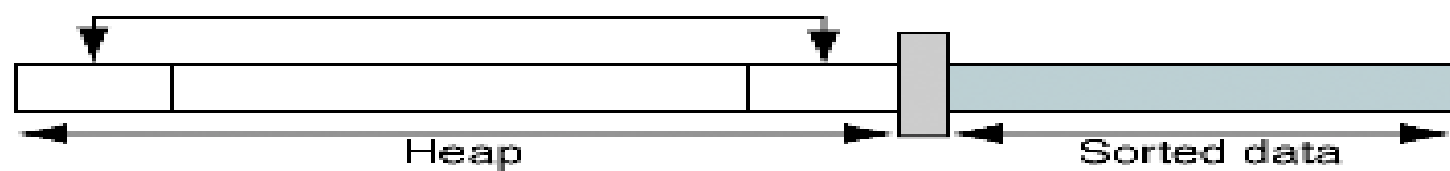


**(a) Heap in its tree form**

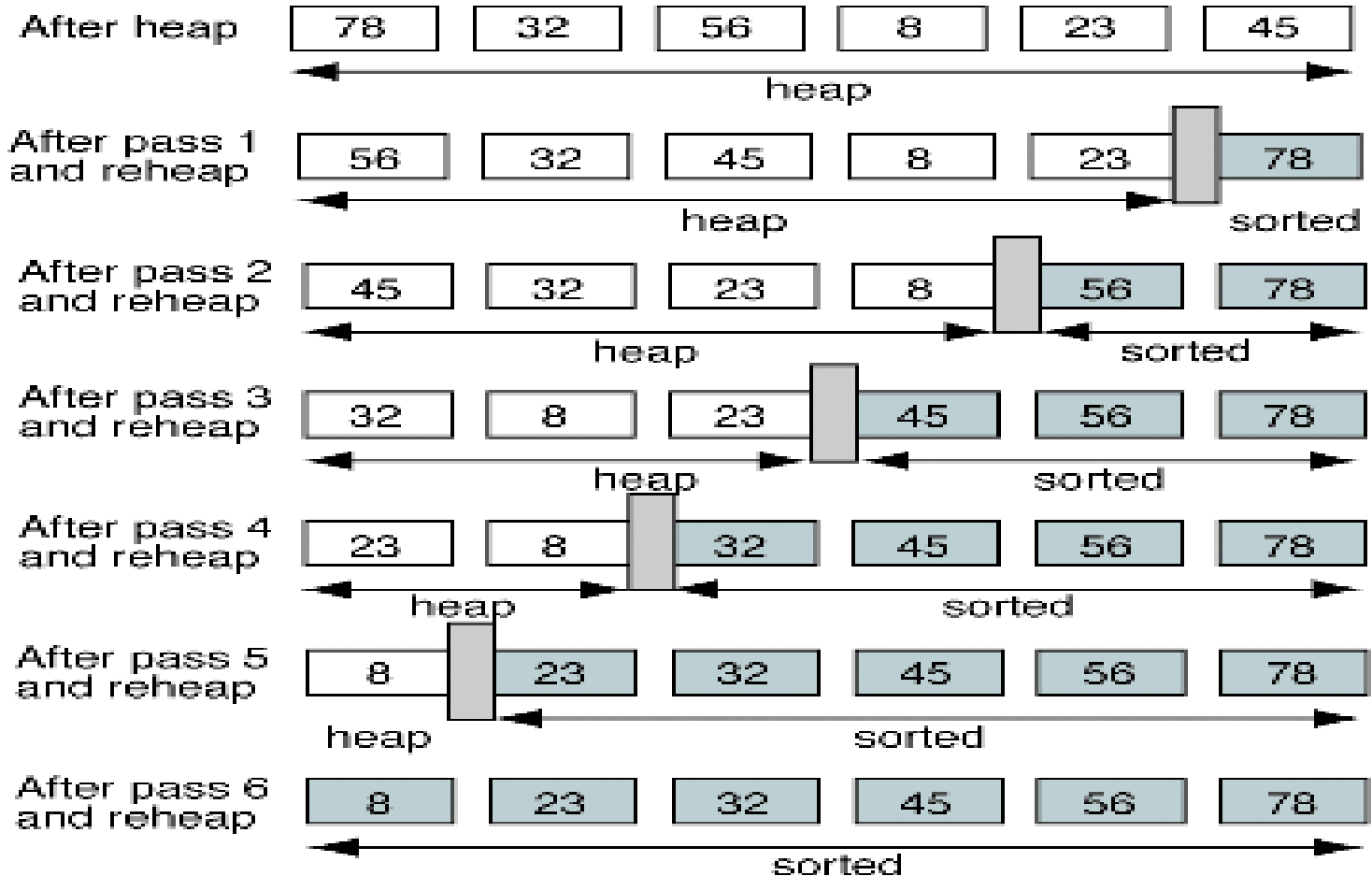


**(b) Heap in its array form**

**FIGURE 12-5** Heap Representations



**(a) Heap sort exchange process**



**(b) Heap sort process**

**FIGURE 12-6** Heap Sort Process

## ALGORITHM 12-2 Heap Sort

```
Algorithm heapSort (heap, last)
Sort an array, using a heap.
    Pre  heap array is filled
        last is index to last element in array
    Post heap array has been sorted
    Create heap
1  set walker to 1
2  loop (heap built)
    1  reheapUp (heap, walker)
    2  increment walker
3  end loop
    Heap created. Now sort it.
4  set sorted to last
5  loop (until all data sorted)
    1  exchange (heap, 0, sorted)
    2  decrement sorted
    3  reheapDown (heap, 0, sorted)
6  end loop
end heapSort
```

| n    | Number of loops    |        |
|------|--------------------|--------|
|      | Straight selection | Heap   |
| 25   | 625                | 116    |
| 100  | 10,000             | 664    |
| 500  | 250,000            | 4482   |
| 1000 | 1,000,000          | 9965   |
| 2000 | 4,000,000          | 10,965 |

**TABLE 12-1** Comparison of Selection Sorts

## PROGRAM 12-1 Selection Sort

```
1  /* ===== selectionSort =====
2  Sorts list [1...last] by selecting smallest element in
3  unsorted portion of array and exchanging it with
4  element at beginning of the unsorted list.
5      Pre  list must contain at least one item
6          last contains index to last list element
7      Post list has been sorted smallest to largest
8  */
9  void selectionSort (int list[ ], int last)
10 {
11     // Local Declarations
12     int smallest;
13     int holdData;
14
15     // Statements
16     for (int current = 0; current < last; current++)
17     {
18         smallest = current;
19         for (int walker = current + 1;
20             walker <= last;
21             walker++)
22             if (list[ walker ] < list[ smallest ])
```

*continued*

## PROGRAM 12-1 Selection Sort (*continued*)

```
23         smallest = walker;
24
25         // Smallest selected: exchange with current
26         holdData      = list[ current ];
27         list[current]  = list[ smallest ];
28         list[smallest] = holdData;
29     } // for current
30     return;
31 } // selectionSort
```

## PROGRAM 12-2 Heap Sort

```
1  /* ===== heapSort =====
2      Sort an array, [list0 .. last], using a heap.
3      Pre  list must contain at least one item
4          last contains index to last element in list
5      Post list has been sorted smallest to largest
6  */
7  void heapSort (int  list[ ], int  last)
8  {
9      // Local Definitions
10     int sorted;
11     int holdData;
12
13     // Statements
14     // Create Heap
15     for (int walker = 1; walker <= last; walker++)
16         reheapUp (list, walker);
17
18     // Heap created. Now sort it.
19     sorted = last;
20     while (sorted > 0)
21     {
22         holdData      = list[0];
23         list[0]       = list[sorted];
24         list[sorted]  = holdData;
```

*continued*

## PROGRAM 12-2 Heap Sort (continued)

```
25         sorted--;  
26         reheapDown (list, 0, sorted);  
27     } // while  
28     return;  
29 } // heapSort  
30  
31 /* ===== reheapUp =====  
32 Reestablishes heap by moving data in child up to  
33 correct location heap array.  
34     Pre  heap is array containing an invalid heap  
35         newNode is index location to new data in heap  
36     Post newNode has been inserted into heap  
37 */
```



## PROGRAM 12-2 Heap Sort (*continued*)

```
38 void reheapUp (int* heap, int newNode)
39 {
40     // Local Declarations
41     int parent;
42     int hold;
43
44     // Statements
45     // if not at root of heap
46     if (newNode)
47     {
48         parent = (newNode - 1) / 2;
49         if ( heap[newNode] > heap[parent] )
50         {
51             // child is greater than parent
52             hold          = heap[parent];
53             heap[parent]  = heap[newNode];
54             heap[newNode] = hold;
55             reheapUp (heap, parent);
56         } // if heap[]
57     } // if newNode
58     return;
59 } // reheapUp
```

## PROGRAM 12-2 Heap Sort *(continued)*

```
60
61  /* ===== reheapDown =====
62     Reestablishes heap by moving data in root down to its
63     correct location in the heap.
64     Pre  heap is an array of data
65          root is root of heap or subheap
66          last is an index to the last element in heap
67     Post heap has been restored
68  */
69  void reheapDown (int* heap, int root, int last)
70  {
71  // Local Declarations
72     int hold;
```

*continued*

## PROGRAM 12-2 Heap Sort (continued)

```
73     int leftKey;
74     int rightKey;
75     int largeChildKey;
76     int largeChildIndex;
77
78     // Statements
79     if ((root * 2 + 1) <= last)
80         // There is at least one child
81         {
82             leftKey = heap[root * 2 + 1];
83             if ((root * 2 + 2) <= last)
84                 rightKey = heap[root * 2 + 2];
85             else
86                 rightKey = -1;
87
88             // Determine which child is larger
```

## PROGRAM 12-2 Heap Sort (continued)

```
89         if (leftKey > rightKey)
90         {
91             largeChildKey    = leftKey;
92             largeChildIndex = root * 2 + 1;
93         } // if leftKey
94     else
95     {
96         largeChildKey    = rightKey;
97         largeChildIndex = root * 2 + 2;
98     } // else
99     // Test if root > larger subtree
100    if (heap[root] < heap[largeChildIndex])
101    {
102        // parent < children
103        hold        = heap[root];
104        heap[root] = heap[largeChildIndex];
105        heap[largeChildIndex] = hold;
106        reheapDown (heap, largeChildIndex, last);
107    } // if root <
108 } // if root
109 return;
110 } // reheapDown
```

# 12-3 Insertion Sorts

*In each pass of an insertion sort, one or more pieces of data are inserted into their correct location in an ordered list. In this section we study two insertion sorts: the straight insertion sort and the shell sort.*

- Straight Insertion Sort
- Shell Sort
- Insertion Sort Efficiency
- Insertion Sort Implementation

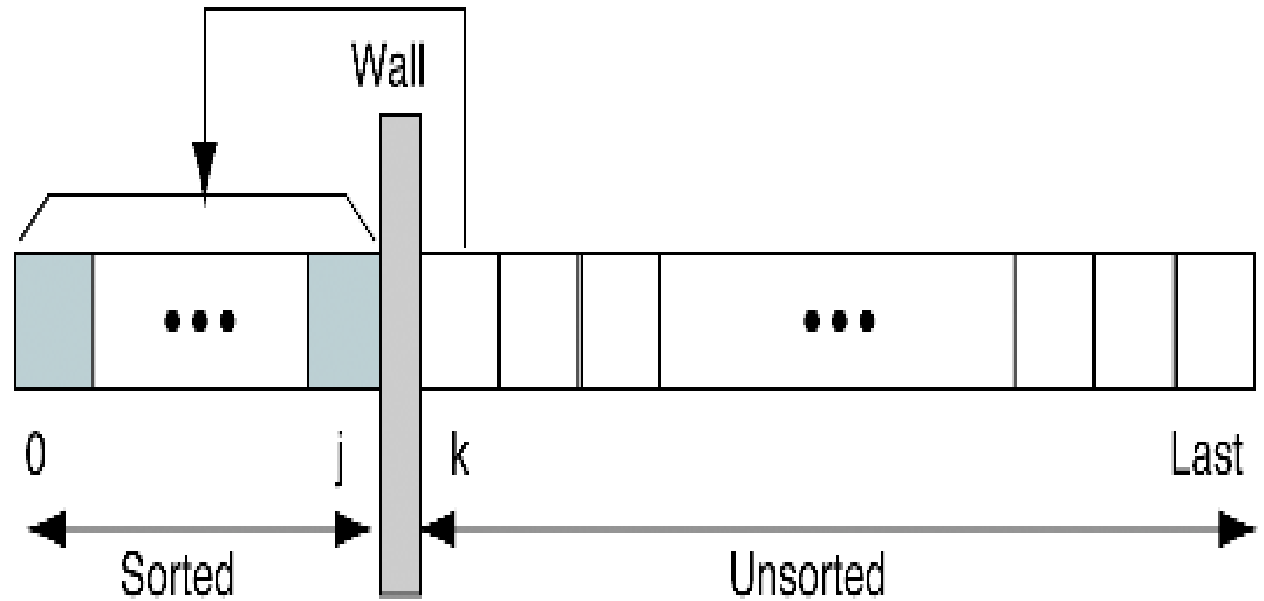
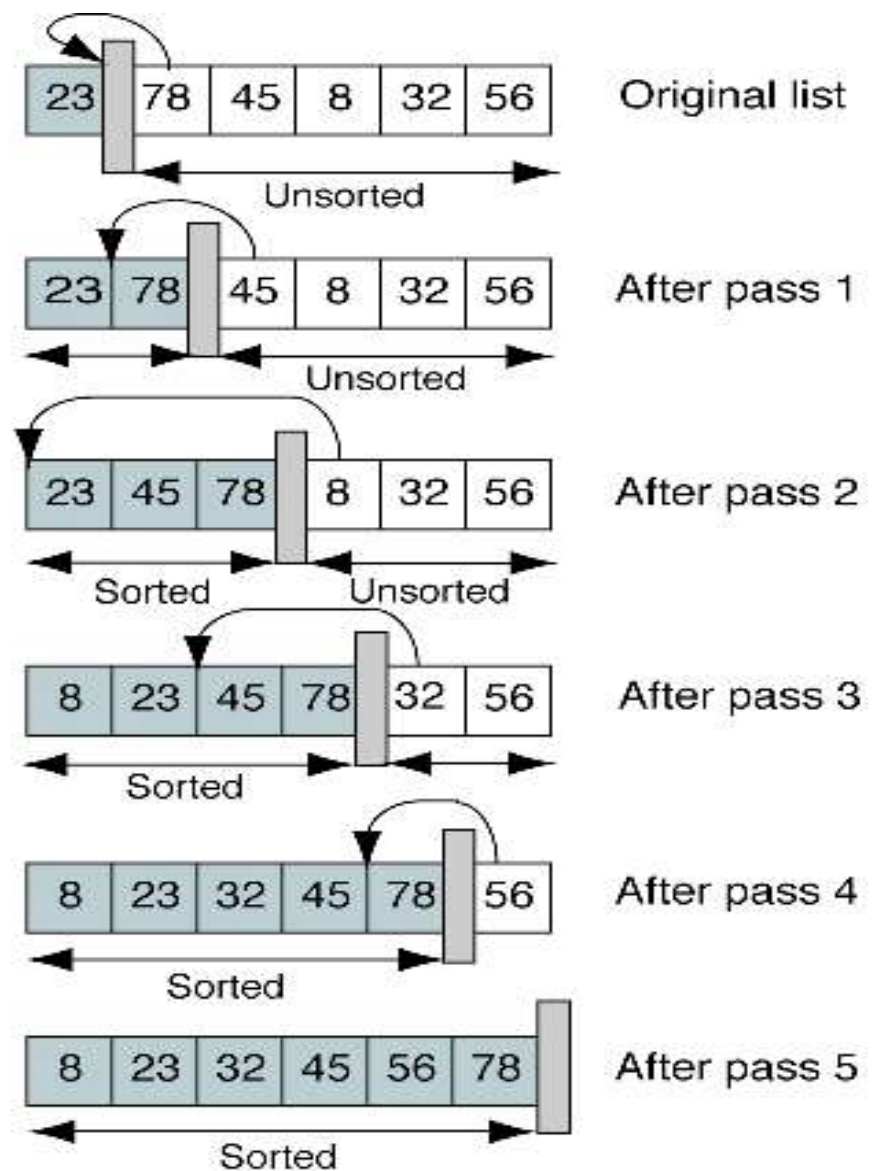


FIGURE 12-7 Insertion Sort Concept

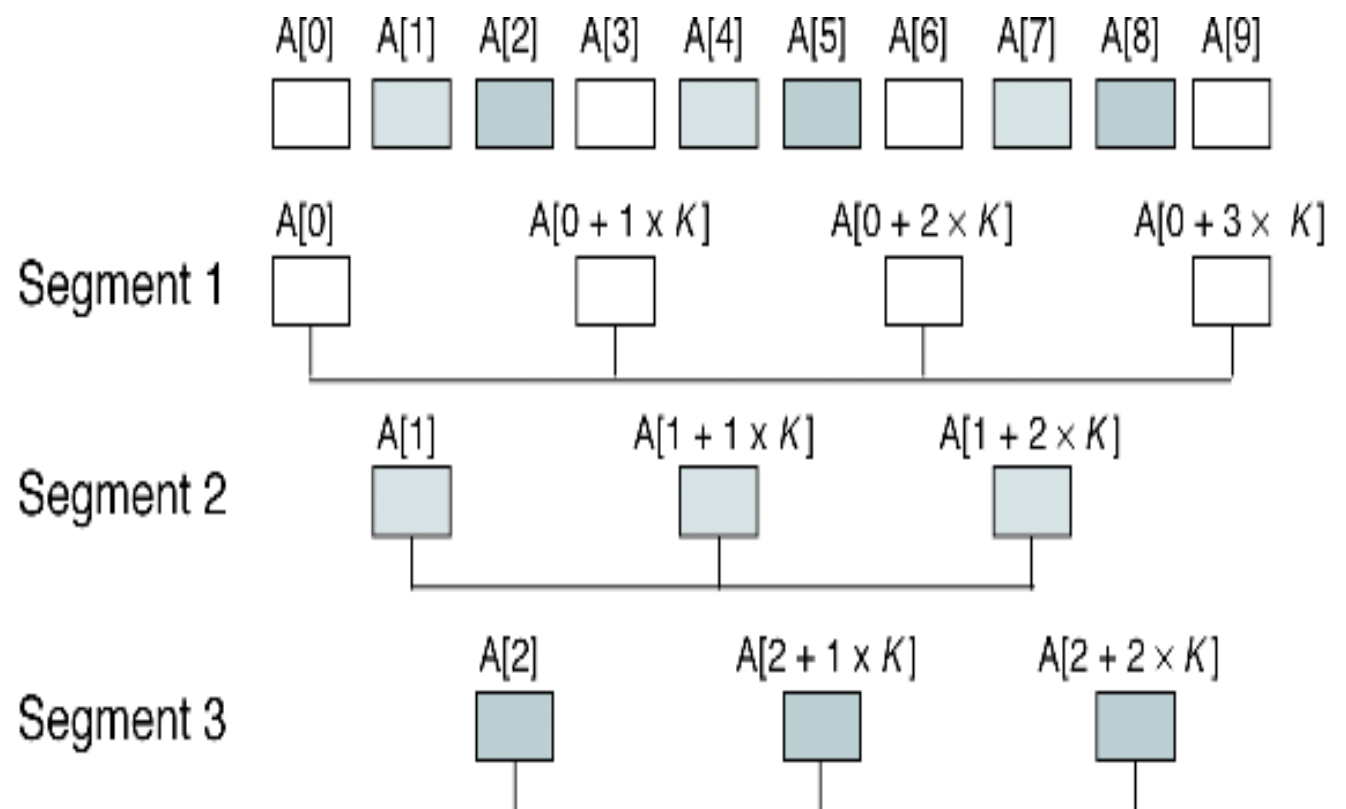


**FIGURE 12-8** Insertion Sort Example

## ALGORITHM 12-3 Straight Insertion Sort

```
Algorithm insertionSort (list, last)
Sort list array using insertion sort. The array is
divided into sorted and unsorted lists. With each pass, the
first element in the unsorted list is inserted into the
sorted list.
    Pre  list must contain at least one element
         last is an index to last element in the list
    Post list has been rearranged
1  set current to 1
2  loop (until last element sorted)
    1  move current element to hold
    2  set walker to current - 1
    3  loop (walker >= 0 AND hold key < walker key)
        1  move walker element right one element
        2  decrement walker
    4  end loop
    5  move hold to walker + 1 element
    6  increment current
3  end loop
end insertionSort
```





**FIGURE 12-9** Segmented Array

FIGURE 12-10 Diminishing Increments in Shell Sort

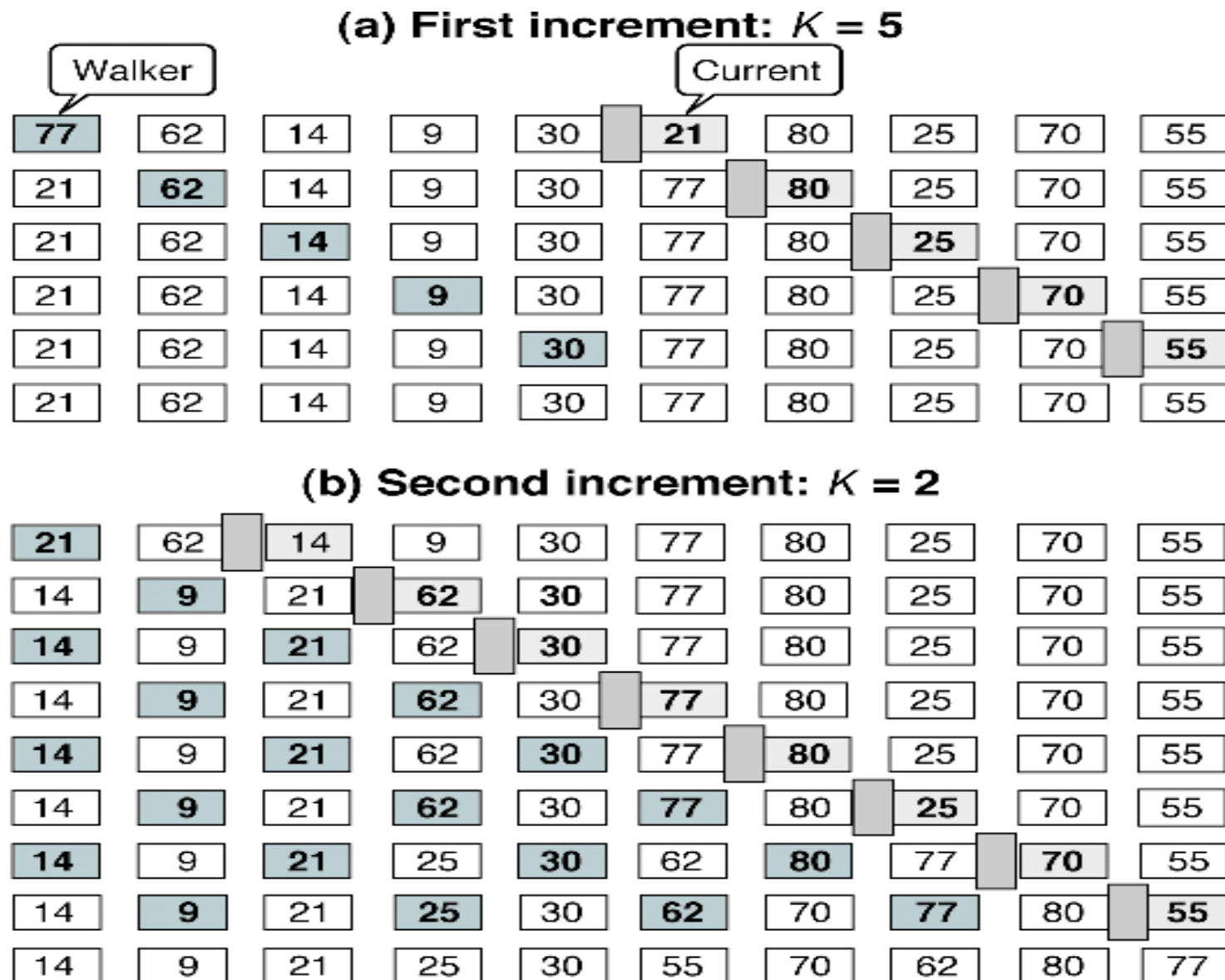


FIGURE 12-10 Diminishing Increments in Shell Sort (continued)

**(c) Third increment:  $K = 1$**

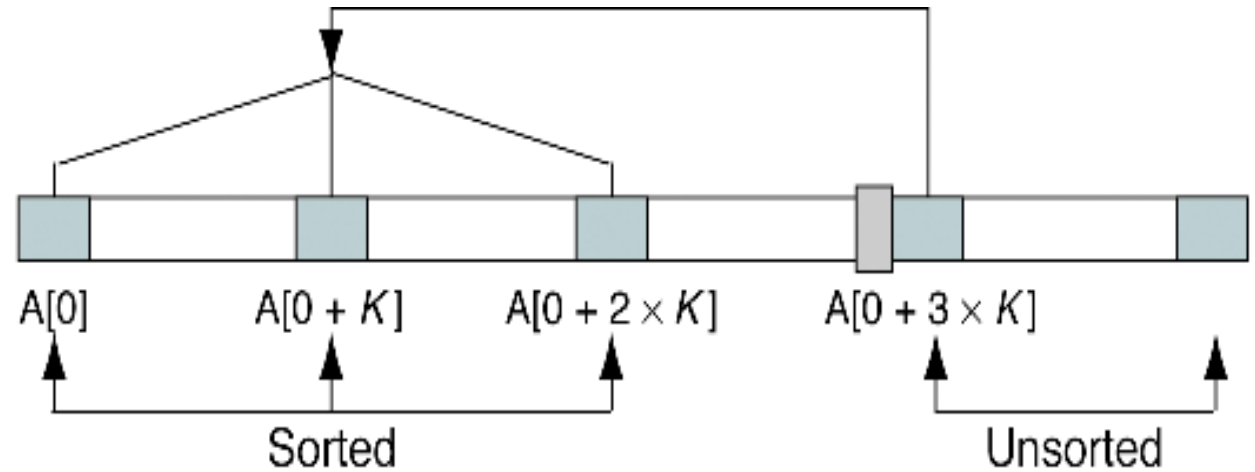
|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 14 | 9  | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 70 | 62 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 62 | 70 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 62 | 70 | 80 | 77 |
| 9  | 14 | 21 | 25 | 30 | 55 | 62 | 70 | 77 | 80 |

**(d) Sorted array**

|   |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|
| 9 | 14 | 21 | 25 | 30 | 55 | 62 | 70 | 77 | 80 |
|---|----|----|----|----|----|----|----|----|----|

## ALGORITHM 12-4 Shell Sort

```
Algorithm shellSort (list, last)
Data in list array are sorted in
place. After the sort, their keys will be in order,
list[0] <= list[1] <= ... <= list[last].
    Pre list is an unordered array of records
        last is index to last record in array
    Post list is ordered on list[i].key
1  set incre to last / 2
    Compare keys "increment" elements apart.
2  loop (incre not 0)
    1  set current to incre
    2  loop (until last element sorted)
        1  move current element to hold
        2  set walker to current - incre
        3  loop (walker >= 0 AND hold key < walker key)
            Move larger element up in list.
            1  move walker element one increment right
            Fall back one partition.
            2  set walker to walker - incre
        4  end loop
        Insert hold record in proper relative location.
    5  move hold to walker + incre element
    6  increment current
    3  end loop
    End of pass--calculate next increment.
    4  set incre to incre / 2
3  end loop
end shellSort
```



**FIGURE 12-11** Ordered Segment in a Shell Sort

| $n$  | Number of loops                          |        |        |
|------|------------------------------------------|--------|--------|
|      | Straight insertion<br>Straight selection | Shell  | Heap   |
| 25   | 625                                      | 55     | 116    |
| 100  | 10,000                                   | 316    | 664    |
| 500  | 250,000                                  | 2364   | 4482   |
| 1000 | 1,000,000                                | 5623   | 9965   |
| 2000 | 4,000,000                                | 13,374 | 10,965 |

**TABLE 12-2** Comparison of Insertion and Selection Sorts

## PROGRAM 12-3 Insertion Sort

```
1  /* ===== insertionSort =====
2      Sort using Insertion Sort. The list is divided into
3      sorted/unordered list. In each pass, first element
4      in unordered list is inserted into sorted list.
5          Pre  list must contain at least one element
6              last contains index to last element in list
7          Post list has been rearranged
8  */
9  void insertionSort (int list[], int last)
10 {
11     // Local Definitions
12     int hold;
13     int walker;
14
15     // Statements
16     for (int current = 1; current <= last; current++)
17     {
18         hold = list[current];
19         for (walker = current - 1;
```

*continued*

### PROGRAM 12-3 Insertion Sort (*continued*)

```
20         walker >= 0 && hold < list[walker];  
21         walker--)  
22         list[walker + 1] = list[walker];  
23  
24         list [walker + 1] = hold;  
25     } // for current  
26     return;  
27 } // insertionSort
```



## PROGRAM 12-4 Shell Sort

```
1  /* List[1], list[2], ..., list[last] are sorted in place
2     so that the keys are ordered, list[1].key <=
3     list[2].key, <= ... <= list[last].key.
4     Pre  list is an unordered array of integers
5         last is index to last element in array
6     Post list is ordered
7  */
8  void shellSort (int list [], int last)
9  {
10 // Local Definitions
11     int hold;
12     int incre;
13     int walker;
14
15 // Statements
16     incre = last / 2;
17     while (incre != 0)
18     {
19         for (int curr = incre; curr <= last; curr++)
20         {
21             hold = list [curr];
22             walker = curr - incre;
23             while (walker >= 0 && hold < list [walker])
24             {
25                 // Move larger element up in list
26                 list [walker + incre] = list [walker];
27                 // Fall back one partition
```

*continued*

## PROGRAM 12-4 Shell Sort (*continued*)

```
28         walker = ( walker - incre );
29     } // while
30     // Insert hold in proper position
31     list [walker + incre] = hold;
32 } // for walk
33 // End of pass--calculate next increment.
34 incre = incre / 2;
35 } // while
36 return;
37 } // shellSort
```

# 12-4 Exchange Sorts

*This section discusses the four basic queue operations. Using diagrammatic figures, it shows how each of them work. It concludes with a comprehensive example that demonstrates each operation.*

- **Bubble Sort**
- **Quick Sort**
- **Exchange Sort Efficiency**
- **Sort Summary**
- **Exchange Sort Implementation**

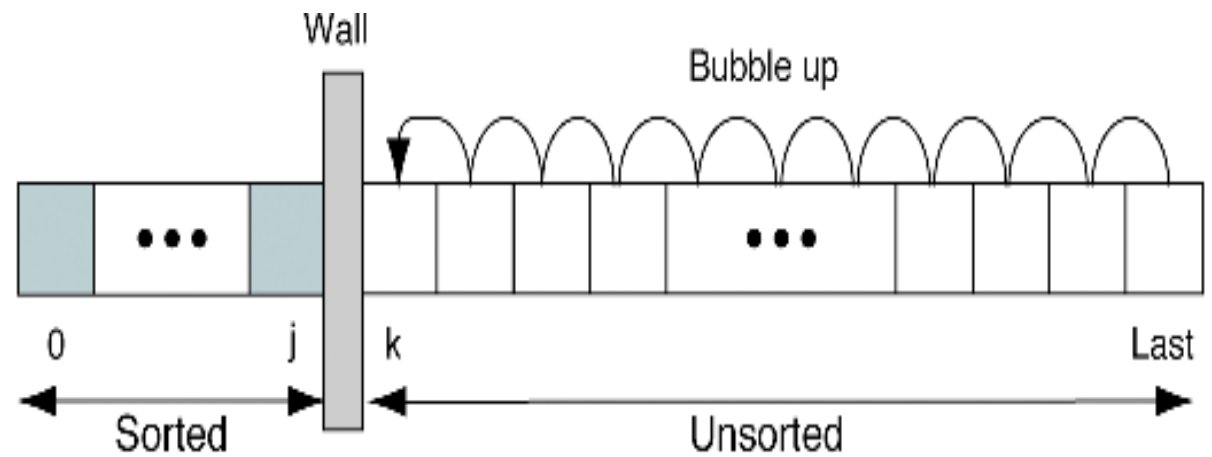
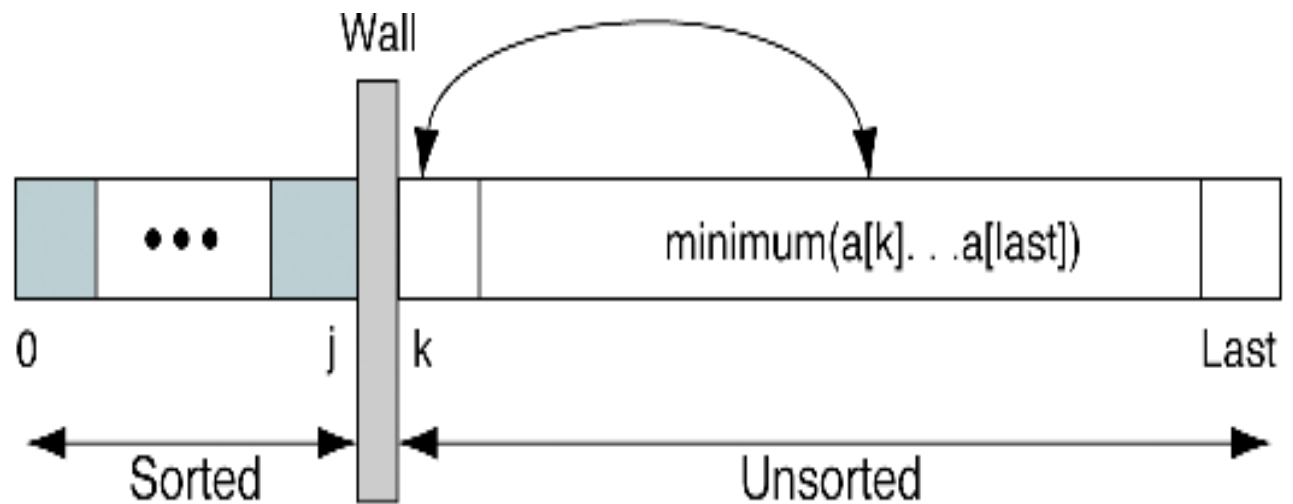


FIGURE 12-12 Bubble Sort Concept



**FIGURE 12-3** Selection Sort Concept

## ALGORITHM 12-5 Bubble Sort

```
Algorithm bubbleSort (list, last)
Sort an array using bubble sort. Adjacent
elements are compared and exchanged until list is
completely ordered.
```

*continued*

## ALGORITHM 12-5 Bubble Sort (*continued*)

```
Pre  list must contain at least one item
      last contains index to last element in the list
Post list has been rearranged in sequence low to high
1  set current to 0
2  set sorted to false
3  loop (current <= last AND sorted false)
    Each iteration is one sort pass.
    1  set walker to last
    2  set sorted to true
    3  loop (walker > current)
        1  if (walker data < walker - 1 data)
            Any exchange means list is not sorted.
            1  set sorted to false
            2  exchange (list, walker, walker - 1)
        2  end if
        3  decrement walker
    4  end loop
    5  increment current
4  end loop
end bubbleSort
```

**After first partitioning**



**After second partitioning**



**After third partitioning**



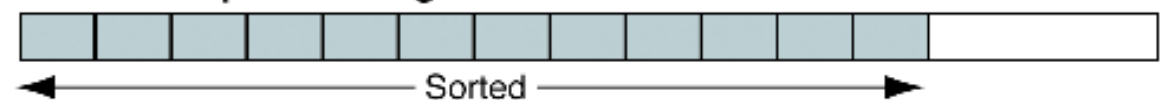
**After fourth partitioning**



**After fifth partitioning**



**After sixth partitioning**



**After seventh partitioning**



**FIGURE 12-14** Quick Sort Partitions



## ALGORITHM 12-6 Quick Sort's Straight Insertion Sort Module

```
Algorithm quickInsertion (list, first, last)
Sort array list using insertion sort. The list is
divided into sorted and unsorted lists. With each pass, the
first element in the unsorted list is inserted into the
sorted list. This is a special version of the insertion
sort modified for use with quick sort.
    Pre  list must contain at least one element
         first is an index to first element in the list
         last is an index to last element in the list
    Post list has been rearranged
1  set current to first + 1
2  loop (until current partition sorted)
    1  move current element to hold
    2  set walker to current - 1
    3  loop (walker >= first AND hold key < walker key)
        1  move walker element one element right
        2  decrement walker
    4  end loop
    5  move hold to walker + 1 element
    6  increment current
3  end loop
end quickInsertion
```

## ALGORITHM 12-7 Median Left

```
Algorithm medianLeft (sortData, left, right)
Find the median value of an array and place it in the first
(left) location.
    Pre    sortData is an array of at least three elements
           left and right are the boundaries of the array
    Post   median value located and placed left
    Rearrange sortData so median value is in middle location.
1  set mid to (left + right) / 2
2  if (left key > mid key)
    1  exchange (sortData, left, mid)
3  end if
4  if (left key > right key)
    1  exchange (sortData, left, right)
5  end if
6  if (mid key > right key)
    1  exchange (sortData, mid, right)
7  end if
    Median is in middle location. Exchange with left.
8  exchange (sortData, left, mid)
end medianLeft
```

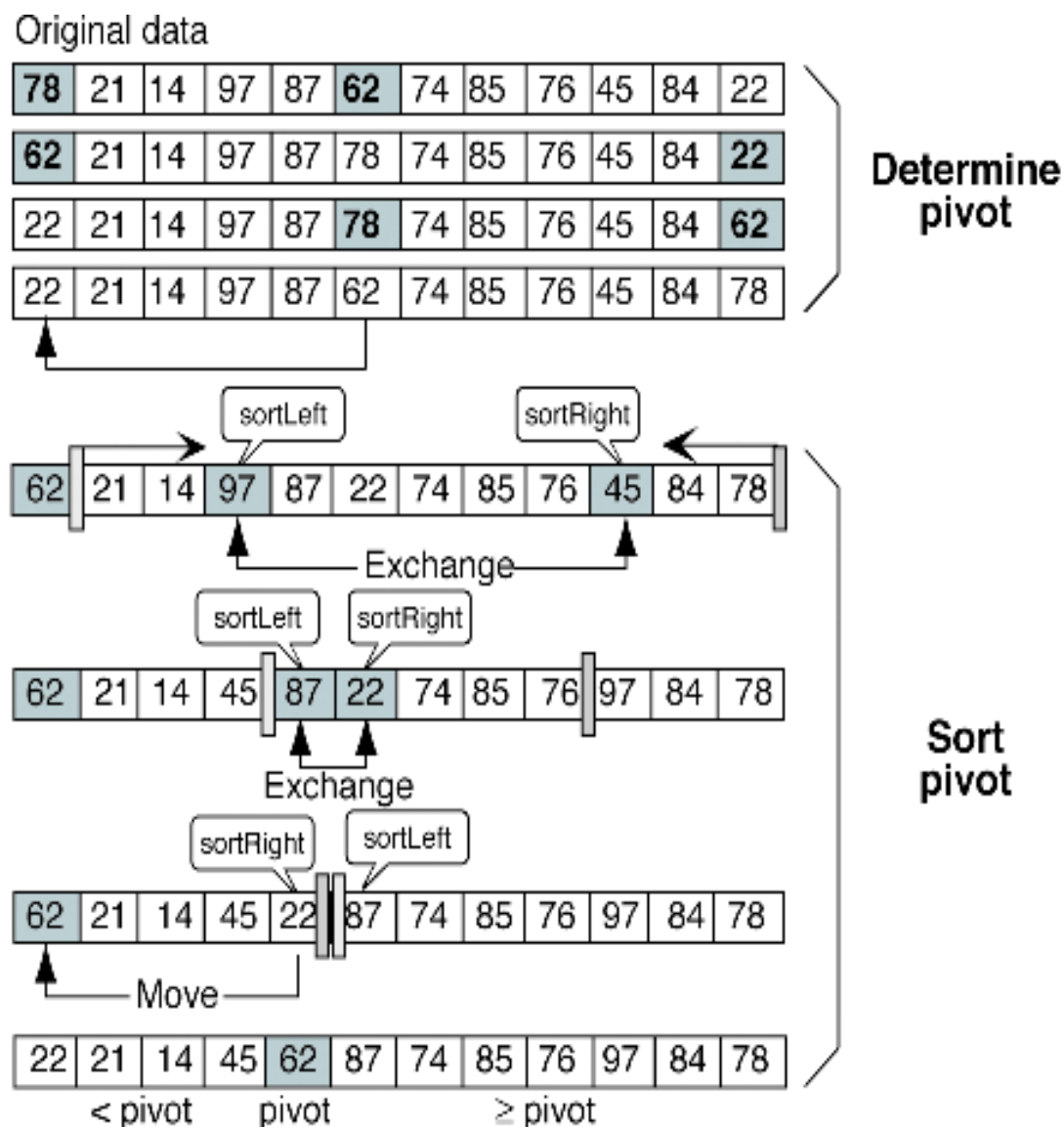


FIGURE 12-15 Quick Sort Pivot

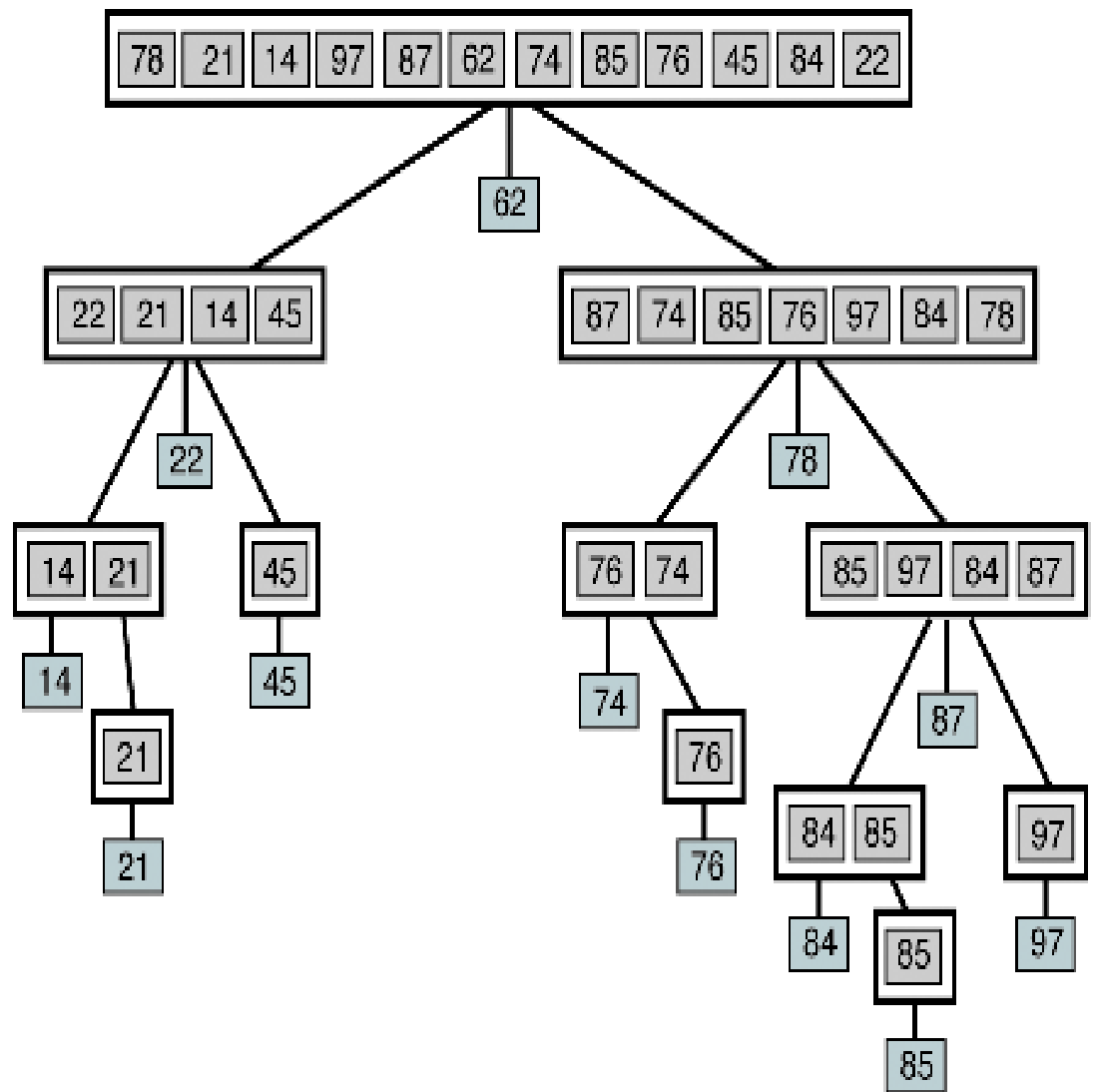


FIGURE 12-16 Quick Sort Operation

## ALGORITHM 12-8 Quick Sort

```
Algorithm quickSort (list, left, right)
An array, list, is sorted using recursion.
  Pre  list is an array of data to be sorted
       left and right identify the first and last
       elements of the list, respectively
  Post list is sorted
1  if ((right - left) > minSize)
    Quick sort
    1  medianLeft (list, left, right)
    2  set pivot    to left element
    3  set sortLeft to left + 1
    4  set sortRight to right
    5  loop (sortLeft <= sortRight)
        Find key on left that belongs on right
```

*continued*



## ALGORITHM 12-8 Quick Sort (*continued*)

```
1  loop (sortLeft key < pivot key)
    1  increment sortLeft
2  end loop
    Find key on right that belongs on left
3  loop (sortRight key >= pivot key)
    1  decrement sortRight
4  end loop
5  if (sortLeft <= sortRight)
    1  exchange(list, sortLeft, sortRight)
    2  increment sortLeft
    3  decrement sortRight
6  end if
6  end loop
    Prepare for next pass
7  move sortLeft - 1 element to left element
8  move pivot element to sortLeft - 1 element
9  if (left < sortRight)
    1  quickSort (list, left, sortRight - 1)
10 end if
11 if (sortLeft < right)
    1  quickSort (list, sortLeft, right)
12 end if
2 else
    1  insertionSort (list, left, right)
3 end if
end quickSort
```

# 12-5 External Sorts

*In external sorting portions of the data may be stored in secondary memory during the sorting process. Included in this section is a discussion of file merging and three external sort approaches-natural, balanced, and polyphase.*

- **Merging Ordered Files**
- **Merging Unordered Files**
- **The Sorting Process**
- **Sort Phase Revisited**

| n    | Number of loops                                          |        |                      |
|------|----------------------------------------------------------|--------|----------------------|
|      | Straight insertion<br>Straight selection<br>Bubble sorts | Shell  | Heap<br>and<br>quick |
| 25   | 625                                                      | 55     | 116                  |
| 100  | 10,000                                                   | 316    | 664                  |
| 500  | 250,000                                                  | 2364   | 4482                 |
| 1000 | 1,000,000                                                | 5623   | 9965                 |
| 2000 | 4,000,000                                                | 13,374 | 10,965               |

**TABLE 12-3** Sort Comparisons



## PROGRAM 12-5 Bubble Sort

```
1  /* ===== bubbleSort =====
2      Sort list using bubble sort. Adjacent elements are
3      compared and exchanged until list is ordered.
4          Pre  list must contain at least one item
5              last contains index to last list element
6          Post list has been sorted in ascending sequence
7  */
8  void bubbleSort (int list [], int last)
9  {
10     // Local Definitions
11     int temp;
12
```

*continued*

## PROGRAM 12-5 Bubble Sort

```
1  /* ===== bubbleSort =====
2      Sort list using bubble sort. Adjacent elements are
3      compared and exchanged until list is ordered.
4          Pre  list must contain at least one item
5              last contains index to last list element
6          Post list has been sorted in ascending sequence
7  */
8  void bubbleSort (int list [], int last)
9  {
10     // Local Definitions
11     int temp;
12
```

*continued*

## PROGRAM 12-5 Bubble Sort (continued)

```
13  // Statements
14  // Each iteration is one sort pass
15  for (int current = 0, sorted = 0;
16      current <= last && !sorted;
17      current++)
18      for (int walker = last, sorted = 1;
19          walker > current;
20          walker--)
21          if (list[ walker ] < list[ walker - 1 ])
22              // Any exchange means list is not sorted
23              {
24                  sorted = 0;
25                  temp      = list[walker];
26                  list[walker] = list[walker - 1];
27                  list[walker - 1] = temp;
28              } // if
29  return;
30 } // bubbleSort
```

## PROGRAM 12-6 Quick Sort

```
1  /* ===== quickSort =====
2      Array, sortData[left..right] sorted using recursion.
3      Pre  sortData is an array of data to be sorted
4          left identifies first element of sortData
5          right identifies last element of sortData
6      Post sortData array is sorted
7  */
8  void quickSort (int sortData[ ], int left, int right)
9  {
10     #define MIN_SIZE 16
11
12     // Local Definitions
13     int sortLeft;
14     int sortRight;
15     int pivot;
16     int hold;
17
```

*continued*

## PROGRAM 12-6 Quick Sort (continued)

```
18  // Statements
19  if ((right - left) > MIN_SIZE)
20  {
21      medianLeft (sortData, left, right);
22      pivot      = sortData [left];
23      sortLeft   = left + 1;
24      sortRight  = right;
25      while (sortLeft <= sortRight)
26      {
27          // Find key on left that belongs on right
28          while (sortData [sortLeft] < pivot)
29              sortLeft = sortLeft + 1;
30          // Find key on right that belongs on left
31          while (sortData[sortRight] >= pivot)
32              sortRight = sortRight - 1;
```

## PROGRAM 12-6 Quick Sort (continued)

```
33         if (sortLeft <= sortRight)
34             {
35                 hold                = sortData[sortLeft];
36                 sortData[sortLeft]  = sortData[sortRight];
37                 sortData[sortRight] = hold;
38                 sortLeft            = sortLeft  + 1;
39                 sortRight           = sortRight - 1;
40             } // if
41         } // while
42         // Prepare for next pass
43         sortData [left]          = sortData [sortLeft - 1];
44         sortData [sortLeft - 1] = pivot;
45         if (left < sortRight)
46             quickSort (sortData, left, sortRight - 1);
47         if (sortLeft < right)
48             quickSort (sortData, sortLeft, right);
49     } // if right
50 else
51     quickInsertion (sortData, left, right);
52     return;
53 } // quickSort
```

## PROGRAM 12-7 Modified Straight Insertion Sort for Quick Sort

```
1  /* ===== quickInsertion =====  
2  Sort data[1..last] using insertion sort. Data is  
3  divided into sorted and unsorted lists. With each  
4  pass, the first element in unsorted list is inserted  
5  into the sorted list. This is a special version of the  
6  insertion sort modified for use with quick sort.  
7      Pre  data must contain at least one element  
8          first is index to first element in data
```

*continued*

## PROGRAM 12-8 Median Left for Quick Sort

```
1  /* ===== medianLeft =====
2  Find the median value of an array,
3  sortData[left..right], and place it in the
4  location sortData[left].
5      Pre  sortData is array of at least three elements
6          left and right are boundaries of array
7          Post median value placed at sortData[left]
8  */
9  void medianLeft (int sortData[], int left, int right)
10 {
11     // Local Definitions
12     int mid;
13     int hold;
14
15     // Statements
16     // Rearrange sortData so median is middle location
17     mid = (left + right) / 2;
```

*continued*



## PROGRAM 12-8 Median Left for Quick Sort *(continued)*

```
18     if (sortData[left] > sortData[mid])
19     {
20         hold                = sortData[left];
21         sortData[left]      = sortData[mid];
22         sortData[mid]       = hold;
23     } // if
24     if (sortData[left] > sortData[right])
25     {
26         hold                = sortData[left];
27         sortData[left]      = sortData[right];
28         sortData[right]     = hold;
29     } // if
30     if (sortData[mid] > sortData[right])
31     {
32         hold                = sortData[mid];
33         sortData[mid]       = sortData[right];
34         sortData[right]     = hold;
35     } // if
36
37     // Median is in middle. Exchange with left
38     hold                = sortData[left];
39     sortData[left]      = sortData[mid];
40     sortData[mid]       = hold;
41
42     return;
43 } // medianLeft
```